Advanced data structures
Course 1
Introduction. Binary search trees.

Ioan Drăgan
ioan.dragan@e-uvt.ro

02.10.2024

Setup

- Course: Ioan Drăgan
  - email: `ioan.dragan@e-uvt.ro`
- Laboratory: Ioan Drăgan
  - email: same as above
- Objectives of the course:
  1. Getting familiar with some of the advanced data structures (ADS) and algorithms that are part of today's state of the art programming;
  2. Be able to recognize the appropriate data structures in order to model and solve in an efficient way problems from real world;
  3. Capability of performing a qualitative analysis for the used algorithms - time complexity analysis.
- Google classroom code: tbd
- Attendance: according to UVT policy

Setup

- Laboratory: implement in C++ applications using data structures and algorithms presented at the course.
- Requirements:
  - ⇒ Read and understand the course material;
  - ⇒ Submit in time the lab work that is proposed.
- Preconditions:
  1. A good understanding of data structures and algorithms studied at Algorithms and datastructures II.
  2. Familiar with programming in C++.
  3. Recommended IDE's: whatever you like.
- Support book:
  - ⇒ T.H. Cormen, C.E. Leiserson, R.L. Rivest, C. Stein: *Introduction to Algorithms*. 3rd Ed. MIT Press. 2009.

Course

1. Intro. Dynamic sets and operations.
2. Red–Black Trees
3. Binary Trees
4. Amortized Analysis
5. Binomial heaps
6. Computational geometry
7. More about mergeable heaps
8. Mid-term exam
9. String matching. The finite automaton approach.
10. Suffix trees. Applications
11. Sorting in linear and sublinear time. Sorting networks. Counting sort
12. Data structures for symbolic computation
13. Recap
14. Final exam

Laborator

1. Binary search trees
2. Red–Black Trees
3. B Trees
4. Binomial heaps
5. Data structures for disjoint sets
6. String matching algorithms
7. Project presentation

Grading

- Course – 50%:
  - ⇒ Mid-term – 25%
  - ⇒ Final exam – 25%
- Laboratory – 50%:
  - ⇒ Lab work – 25%
  - ⇒ Project – 20%
  - ⇒ Activity during labs – 5%

Minimal knowledge for passing grade:

- Write pseoudocode for a simple data structure operation that was discussed during the course
- Correctly identify the computational complexity of an operation or algorithm
- Correctly apply a simple algorithm or operation

Dynamic sets

Elements of a dynamic set

A dynamic set is a collection of objects that may grow, shrink, or otherwise change over time.

- Objects have fields
- A key is a field that uniquely identifies an object.
  - In all implementations of dynamic sets presented in this lecture, keys are assumed to be totally ordered.
- Operations on a dynamic set are grouped into two categories:
  1. **Queries:** they simply return information about the set. Exemple: SEARCH, MINIM, MAXIM, SUCCESOR, PREDECESOR
  2. **Modifying operations:** INSERARE, ȘTERGERE

Operations on a dynamic set S

1. search(S, k): return a pointer p to an element in S such that $p\text{->}key = k$, or Nil if no such element exists.
2. minimum(S): return a pointer p to an element in S whose key is minimum, or Nil if S has no elements.
3. maximum(S): return a pointer p to an element in S whose key is maximum, or Nil if S has no elements.
4. successor(S, x): returns the next element larger than x in S, or Nil if x is the maximum element.
5. successor(S, x): returns the next element smaller than x in S, or Nil if x is the minimum element.
6. insert(S, p): increase S with the element pointed by p. If p is Nil, do nothing.
7. delete(S, p): remove the element pointed by p from S. If p is Nil, do nothing.

Operations 1-5 are queries; operations 6-7 are modifying operations.

Dynamic sets

Implementation issues

- The complexity of operations of a dynamic set is measured in terms of its size n = number of elements.
- Different implementations of dynamic sets vary by the runtime complexity of their operations
  - The choice of an implementation depends on the operations we perform most often
  - Typical examples:
    - Binary search trees
    - Red-Black trees
    - B-Trees
    - Binomial heaps
    - Fibonacci heaps
    - ...

Binary search trees (recap)

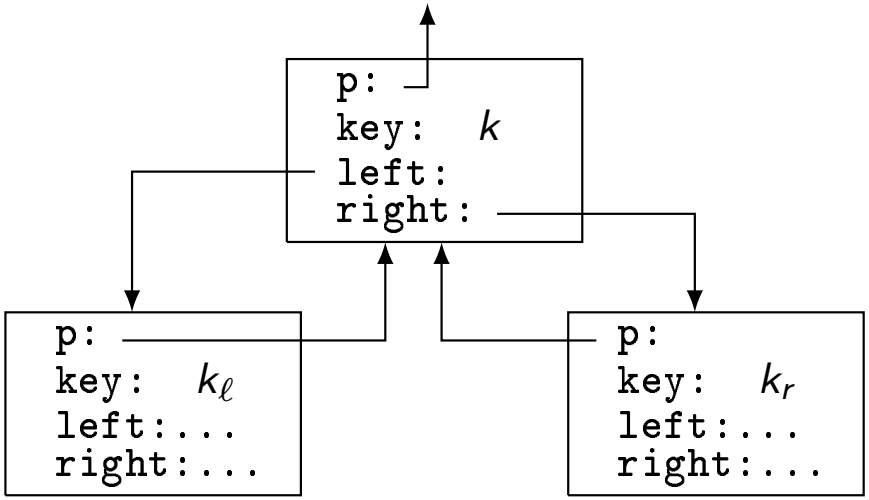
- Relevant information is stored in the nodes of a binary tree
  - ⇒ every non-leaf node has a left and a right child
- Every node has an unique key, that is used to identify the node
- The finding of a node with a particular key must be fast

The keys are distributed in a special way:

⇒ For every node, its key is larger than the keys of the nodes to its left, and smaller than the keys of the nodes to its right.

Implementation

```
struct Node {
    int key;      // key
    Node *p;      // pointer to parent
    Node *left;   // pointer to left child
    Node *right;  // pointer la right child
    ...           // satellite data
};
struct BSTree {
   Node *root;    // pointer to node at root position
   ...            // operations on trees
};
```



Leaf nodes are empty: they contain no data.

A pointer to a leaf node is NIL (0)

Requirement: $k_\ell \leq k \leq k_r$

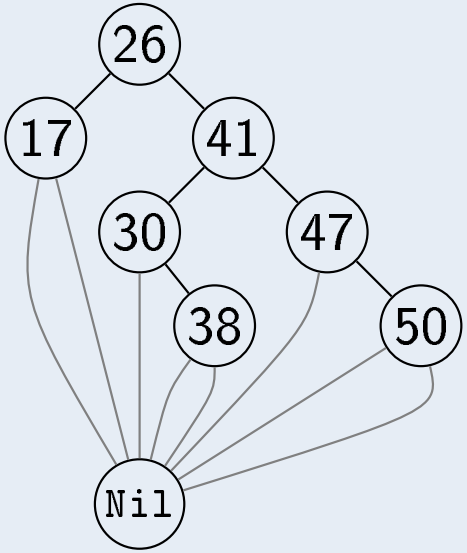
If n is root node then n.p == 0.

Implementation issues and diagrammatic representations

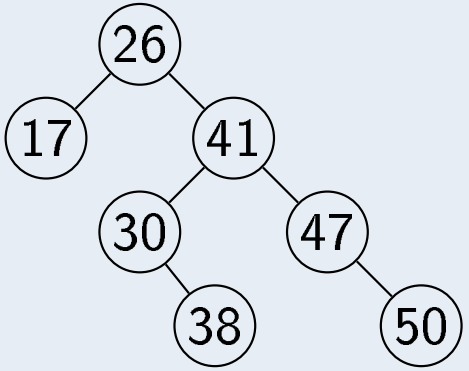
In OOP (e.g., C++), there are two ways of representing leaf nodes:

1. with a single sentinel node Nil, which is also the root's parent
2. with the NULL pointer

Graphical representation



OR



Inserting into a binary search tree

T.insert(z) where

T : binary search tree
z : pointer to node with z->key $= v$, z->left $=$ z->right $= Nil$

Effect: T and z are modified such that z is inserted in the corresponding position in T.

```
insert(Node* z) // method of the class BSTree
y = 0
x =root
while x ≠ 0
  y := x
  if z->key < x->key
    x = x->left
  else x = x->right
z->p = y
if y == 0
  root = z
else if (z->key < y->key)
  y->left = z
else y->right = z
```

Deleting from a binary search tree

T.del(z) deleting z from the binary search tree T

```
del(Node* z) // classical method BSTree
 1 if z->left == 0 or z->right == 0
 2   y := z
 3 else y =successor(z)
 4 if y->left ≠ 0
 5   x = y->left
 6 else x = y->right
 7 if x ≠ 0
 8   x->p = y->p
 9   if y->p == 0
10     root = x
11   else if y == y->p->left
12          y->p->left = x
13        else y->p->right = x
14 if y ≠ z
15   z->key = y->key
16   if y has other fields, copy them in z
17 return y
```

Time complexity of operations

Assumption: S is a instance of `BSTree`, which represents a binary search tree with n nodes:

Operation	Average case	Worst case
S.`search`(k)	$O(\log_2 n)$	$O(n)$
S.`minimum`()	$O(\log_2 n)$	$O(n)$
S.`maximum`()	$O(\log_2 n)$	$O(n)$
S.`successor`(x)	$O(\log_2 n)$	$O(n)$
S.`predecessor`(x)	$O(\log_2 n)$	$O(n)$
S.`insert`(x)	$O(\log_2 n)$	$O(n)$
S.`del`(x)	$O(\log_2 n)$	$O(n)$

Time complexity of operations

- Average case time complexity is $O(\log n)$ for all operations.
- Worst case time complexity is $O(n)$ for all operations.

Question: Can we improve the worst case time complexity of binary search trees without many changes?

Time complexity of operations

- Average case time complexity is $O(\log n)$ for all operations.
- Worst case time complexity is $O(n)$ for all operations.

Question: Can we improve the worst case time complexity of binary search trees without many changes?

Answer: Yes, using Red-Black trees

References

- Chapters 12 (Binary Search Trees) and 3 (Growth of Functions) from the reference book.
  - ⇒ T.H. Cormen, C.E. Leiserson, R.L. Rivest, C. Stein: *Introduction to Algorithms*. 3rd Ed. MIT Press. 2009.

- Next course:
  - ⇒ Red-Black trees – Chapter 13 (Red-Black Trees)